AI ASSISTED CODING- 11.3

M.AKHIL REDDY

2303A52315

BATCH 45

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
   - o Add a contact
   - o Search for a contact
   - o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
   - o Insertion efficiency
   - o Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

PROMPT:

Generate a Python program for a Contact Manager system.

Requirements:

1. Implement using an array (Python list) and a Linked List.

2. Each contact should contain name and phone number.

3. Provide operations:

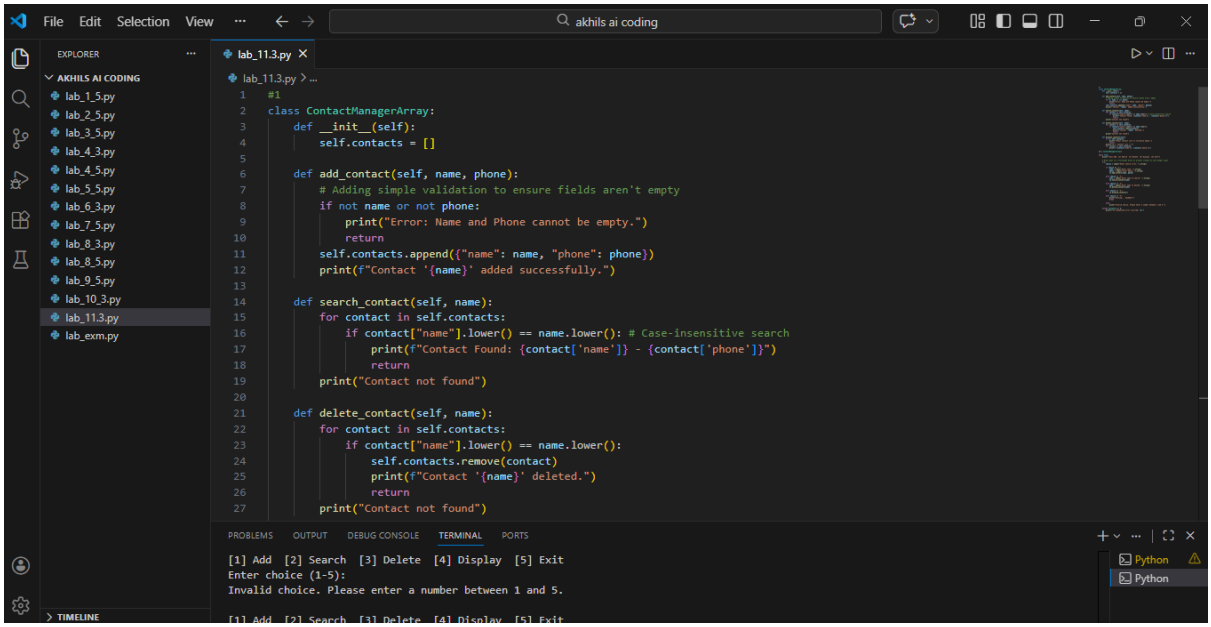
- Add Contact
- Search Contact
- Delete Contact
- Display Contacts

4. Use classes and make the program menu-driven with user input.

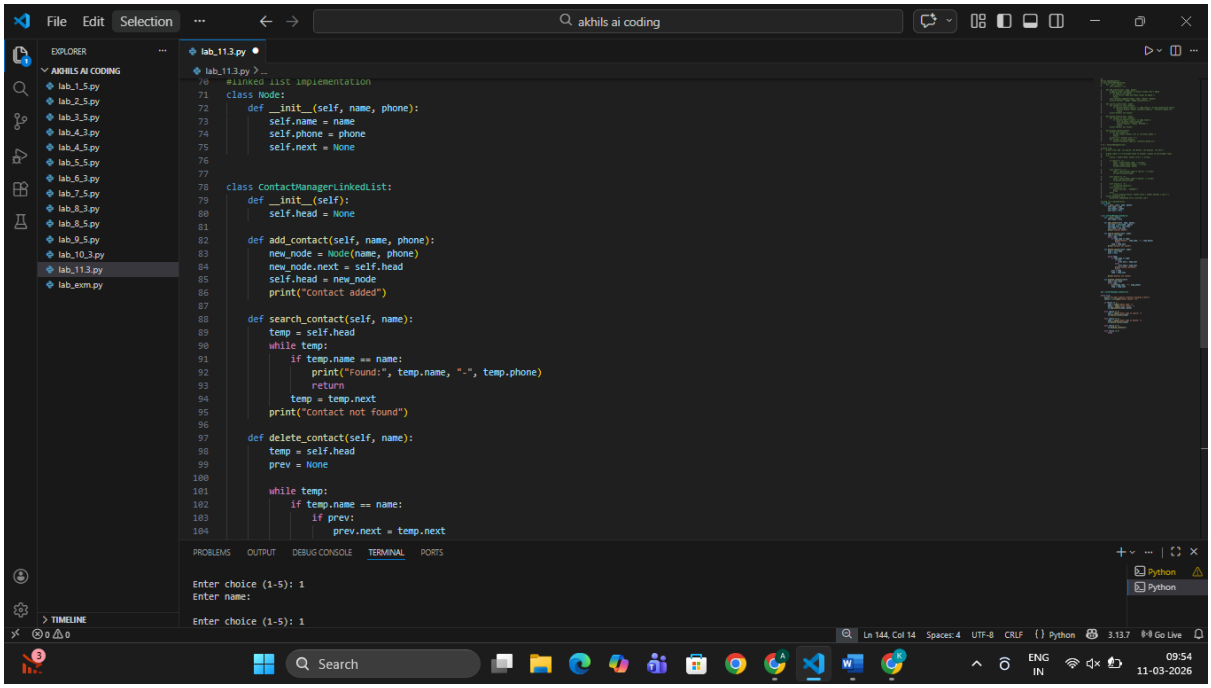
5. Add comments explaining the code.

OUTPUT:

#Array Implementation



#Linked List Implementation



Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

PROMPT:

Generate a Python program for a Library Book Request System.

Requirements:

1. Implement a Queue (FIFO) using Python list.

2. Implement a Priority Queue where faculty requests are served before students.

3. Include methods:

   enqueue()

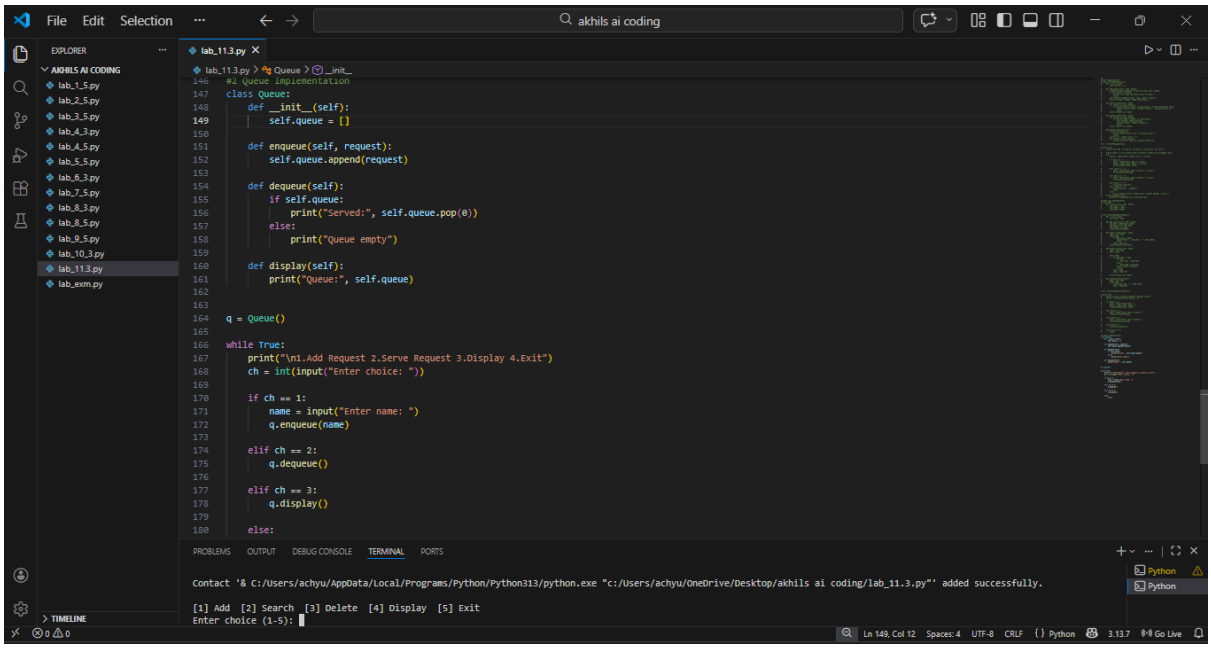
   dequeue()

   display()

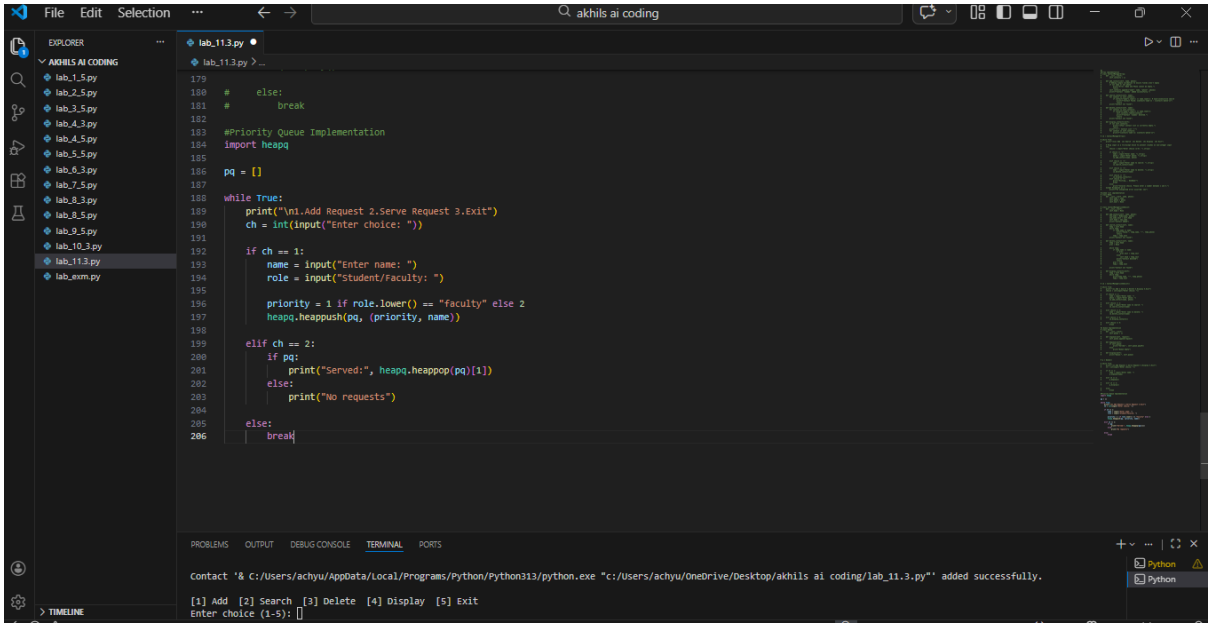
4. Program should accept user input for requests.

OUTPUT:

#Queue Implementation



#Priority Queue Implementation



Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Expected Outcome

• Functional stack-based ticket management system.

• Clear demonstration of LIFO behavior.

PROMPT:

Generate a Python program implementing a Stack for IT Help Desk tickets.

Requirements:

1. Implement push(), pop(), peek()

2. Include methods to check stack empty and full.

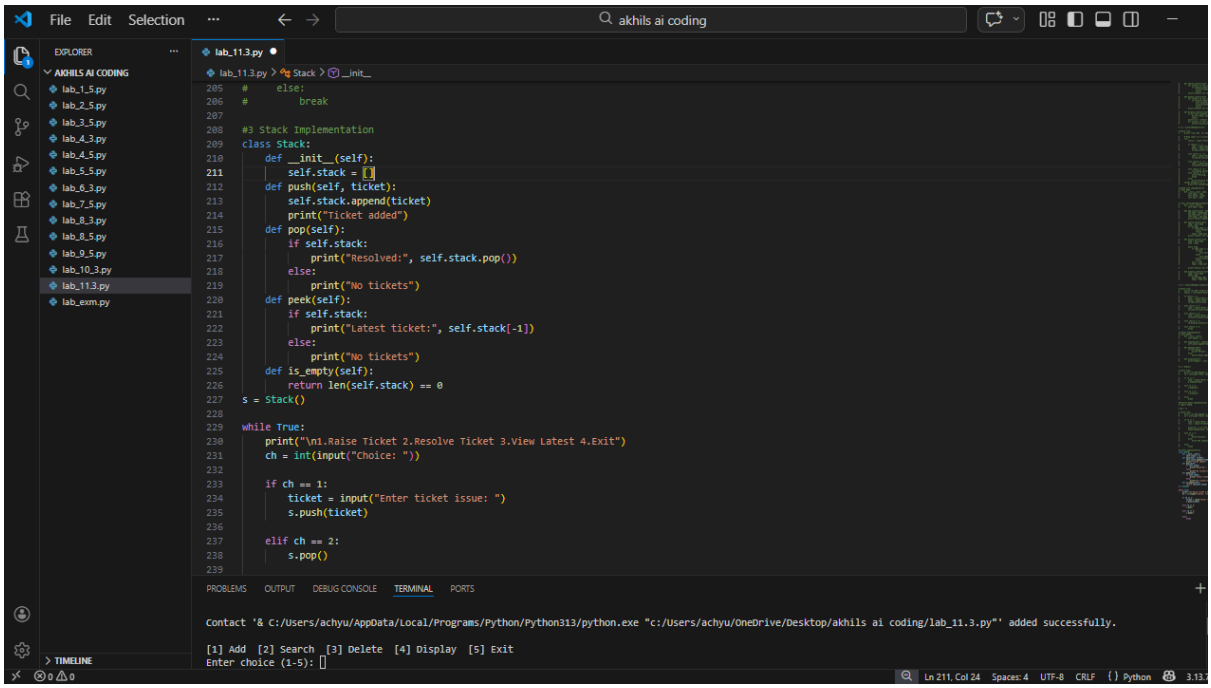
3. Use list to implement stack.

4. Simulate raising and resolving tickets.

5. Use menu driven input.

OUTPUT:

#Stack Implementation



Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Expected Outcome

• Collision handling using chaining

• Well-commented methods

PROMPT:

Generate a Python Hash Table implementation with collision handling using chaining.

Requirements:

1. Methods:

   insert(key,value)

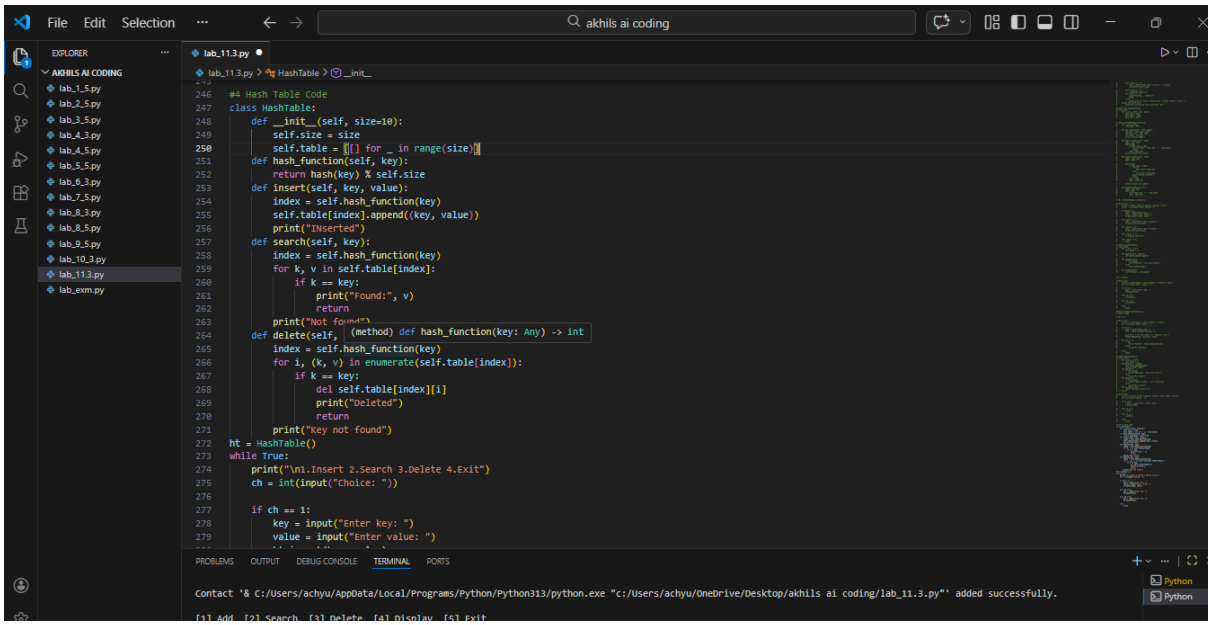
   search(key)

   delete(key)

2. Use lists for chaining.

3. Add comments explaining hash function and collisions.

OUTPUT:



Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following

features:

• Student Attendance Tracking

• Event Registration System

• Library Book Borrowing

• Bus Scheduling System

• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

• Mapping table: Feature → Data Structure → Justification

• One fully working Python implementation

PROMPT:

Create a Python program for a Cafeteria Order Queue system.

Requirements:

1. Use Queue data structure.

2. Allow placing order and serving order.

3. Use menu driven program.

4. Display current orders.

OUTPUT:

File Edit Selection View ··· akhils ai coding

EXPLORER

AKHILS AI CODING

- lab_1_5.py
- lab_2_5.py
- lab_3_5.py
- lab_4_3.py
- lab_4_5.py
- lab_5_5.py
- lab_6_3.py
- lab_7_5.py
- lab_8_3.py
- lab_8_5.py
- lab_9_5.py
- lab_10_3.py
- lab_11.3.py
- lab_exm.py

lab_11.3.py

lab_11.3.py > ...

```python
#          break

#5
from collections import deque

orders = deque()

while True:
    print("\n1.Place Order 2.Serve Order 3.Display Orders 4.Exit")
    ch = int(input("Choice: "))

    if ch == 1:
        order = input("Enter order name: ")
        orders.append(order)

    elif ch == 2:
        if orders:
            print("Serving:", orders.popleft())
        else:
            print("No orders")

    elif ch == 3:
        print("Orders:", list(orders))

    else:
        break
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Contact '& C:/Users/achyu/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/achy
```